F Y E O

Security Code Review of XRPL Sponsored Fees and Reserves

Ripple

February 2026
Version 1.0

Presented by:
FYEO Inc.
PO Box 147044
Lakewood CO 80214
United States

Security Level
Public

Table of Contents

Executive Summary....2
- Overview....2
- Key Findings....2
- Scope and Rules of Engagement....3

Technical Analyses and Findings....6
- Findings....7
- The Classification of vulnerabilities....8
- Technical Analysis....9
- Conclusion....9

Technical Findings....10
- General Observations....10
- SponsorshipSet uses sfFeeAmount without presence check....11
- SponsorshipTransfer hardcodes sfSponsor breaking trustlines....12
- Wrong ledger entry receives sponsor in MPToken escrow path....13
- removeEmptyHolding doesn't retrieve sponsor from trustline being deleted....14
- Payment with tfSponsorCreatedAccount does not validate reserve....15
- Potential unsigned integer overflow in adjustOwnerCount....17
- Trustline sponsor field default parameter is dangerous....18
- sfSponsorFlags allowed without sfSponsor field....19
- Code Quality Concerns....20
- NFTokenUtils removeToken has unused sponsor variable....21

Our Process....22
- Methodology....22
- Kickoff....22
- Ramp-up....22
- Review....23
- Code Safety....23
- Technical Specification Matching....23
- Reporting....24
- Verify....24

Additional Note....24

Executive Summary

Overview

Ripple engaged FYEO Inc. to perform a Security Code Review of XRPL Sponsored Fees and Reserves.

The assessment was conducted remotely by the FYEO Security Team. Testing took place on February 02 - February 13, 2026, and focused on the following objectives:

- To provide the customer with an assessment of their overall security posture and any risks that were discovered within the environment during the engagement.
- To provide a professional opinion on the maturity, adequacy, and efficiency of the security measures that are in place.
- To identify potential issues and include improvement recommendations based on the results of our tests.

This report summarizes the engagement, tests performed, and findings. It also contains detailed descriptions of the discovered vulnerabilities, steps the FYEO Security Team took to identify and validate each issue, as well as any applicable recommendations for remediation.

Key Findings

The following issues have been identified during the testing period. These should be prioritized for remediation to reduce the risk they pose:

- FYEO-XRPL-01 – SponsorshipSet uses sfFeeAmount without presence check
- FYEO-XRPL-02 – SponsorshipTransfer hardcodes sfSponsor breaking trustlines
- FYEO-XRPL-03 – Wrong ledger entry receives sponsor in MPToken escrow path
- FYEO-XRPL-04 – removeEmptyHolding doesn't retrieve sponsor from trustline being deleted
- FYEO-XRPL-05 – Payment with tfSponsorCreatedAccount does not validate reserve
- FYEO-XRPL-06 – Potential unsigned integer overflow in adjustOwnerCount
- FYEO-XRPL-07 – Trustline sponsor field default parameter is dangerous
- FYEO-XRPL-08 – sfSponsorFlags allowed without sfSponsor field
- FYEO-XRPL-09 – Code Quality Concerns
- FYEO-XRPL-10 – NFTokenUtils removeToken has unused sponsor variable

Based on our review process, we conclude that the reviewed code implements the documented functionality.

Scope and Rules of Engagement

The FYEO Review Team performed a Security Code Review of XRPL Sponsored Fees and Reserves. The following table documents the targets in scope for the engagement. No additional systems or resources were in scope for this assessment.

The source code was supplied through a public repository at https://github.com/XRPLF/rippled/ with the commit hash e1aee43359be003019997e6ded3bedaa934faba7.

Remediations were provided with the commit hash 99618877e463695e5ac8092994d424124e7bf459.

Files included in the code review

`rippled/`
`├── include`
`│  └── xrpl`
`│  ├── ledger`
`│  │  └── View.h`
`│  └── protocol`
`│  ├── Fees.h`
`│  ├── Indexes.h`
`│  ├── LedgerFormats.h`
`│  ├── Sponsor.h`
`│  ├── TxFlags.h`
`│  ├── detail`
`│  │  ├── ledger_entries.macro`
`│  │  ├── permissions.macro`
`│  │  ├── sfields.macro`
`│  │  └── transactions.macro`
`│  └── jss.h`
`└── src`
`  ├── libxrpl`
`  │  ├── ledger`
`  │  │  ├── CredentialHelpers.cpp`
`  │  │  └── View.cpp`
`  │  └── protocol`
`  │  ├── Indexes.cpp`
`  │  ├── InnerObjectFormats.cpp`
`  │  ├── LedgerFormats.cpp`
`  │  ├── STTx.cpp`
`  │  ├── TER.cpp`
`  │  └── TxFormats.cpp`

Files included in the code review
`└── xrpld`
`├── app`
`│  ├── paths`
`│  │  └── AMMOffer.h`
`│  └── tx`
`│  └── detail`
`│  ├── AMMClawback.cpp`
`│  ├── AMMCreate.cpp`
`│  ├── AMMDeposit.cpp`
`│  ├── AMMWithdraw.cpp`
`│  ├── AMMWithdraw.h`
`│  ├── Batch.cpp`
`│  ├── CancelCheck.cpp`
`│  ├── CashCheck.cpp`
`│  ├── CreateCheck.cpp`
`│  ├── CreateOffer.cpp`
`│  ├── CreateTicket.cpp`
`│  ├── Credentials.cpp`
`│  ├── DID.cpp`
`│  ├── DelegateSet.cpp`
`│  ├── DeleteAccount.cpp`
`│  ├── DeleteOracle.cpp`
`│  ├── DepositPreauth.cpp`
`│  ├── Escrow.cpp`
`│  ├── InvariantCheck.cpp`
`│  ├── InvariantCheck.h`
`│  ├── LoanBrokerCoverClawback.cpp`
`│  ├── LoanBrokerCoverDeposit.cpp`
`│  ├── LoanBrokerDelete.cpp`
`│  ├── LoanBrokerSet.cpp`
`│  ├── LoanDelete.cpp`
`│  ├── LoanManage.cpp`
`│  ├── LoanPay.cpp`
`│  ├── LoanSet.cpp`
`│  ├── MPTokenAuthorize.cpp`
`│  ├── MPTokenIssuanceCreate.cpp`
`│  ├── MPTokenIssuanceCreate.h`
`│  ├── MPTokenIssuanceDestroy.cpp`
`│  ├── NFTokenAcceptOffer.cpp`
`│  ├── NFTokenCreateOffer.cpp`
`│  ├── NFTokenMint.cpp`
`│  ├── NFTokenUtils.cpp`
`│  ├── NFTokenUtils.h`

Files included in the code review
`│ ├── PayChan.cpp`
`│ ├── Payment.cpp`
`│ ├── PermissionedDomainDelete.cpp`
`│ ├── PermissionedDomainSet.cpp`
`│ ├── SetAccount.cpp`
`│ ├── SetOracle.cpp`
`│ ├── SetOracle.h`
`│ ├── SetSignerList.cpp`
`│ ├── SetTrust.cpp`
`│ ├── SponsorshipSet.cpp`
`│ ├── SponsorshipSet.h`
`│ ├── SponsorshipTransfer.cpp`
`│ ├── SponsorshipTransfer.h`
`│ ├── Transactor.cpp`
`│ ├── Transactor.h`
`│ ├── VaultClawback.cpp`
`│ ├── VaultCreate.cpp`
`│ ├── VaultDelete.cpp`
`│ ├── VaultDeposit.cpp`
`│ ├── VaultWithdraw.cpp`
`│ ├── XChainBridge.cpp`
`│ └── applySteps.cpp`
`└── rpc`
`└── handlers`
`├── AccountObjects.cpp`
`├── LedgerEntry.cpp`
`└── Simulate.cpp`

Table 1: Scope

Technical Analyses and Findings

During the Security Code Review of XRPL Sponsorship System, we discovered:

- 4 findings with HIGH severity rating.
- 1 finding with MEDIUM severity rating.
- 3 findings with LOW severity rating.
- 2 findings with INFORMATIONAL severity rating.

The following chart displays the findings by severity.

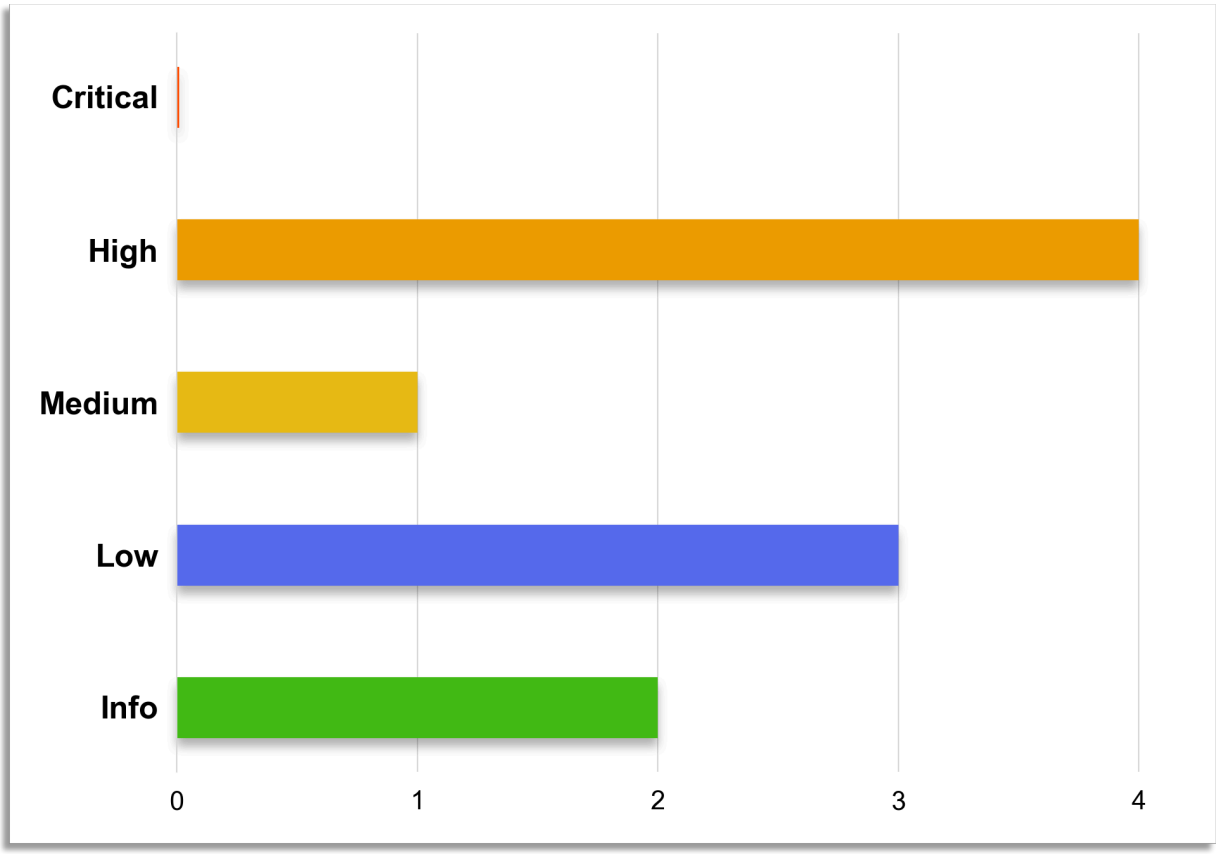

Figure 1: Findings by Severity

Findings

The *Findings* section provides detailed information on each of the findings, including methods of discovery, explanation of severity determination, recommendations, and applicable references.

The following table provides an overview of the findings.

Finding #	Severity	Description	Status
FYEO-XRPL-01	High	SponsorshipSet uses sfFeeAmount without presence check	Remediated
FYEO-XRPL-02	High	SponsorshipTransfer hardcodes sfSponsor breaking trustlines	Remediated
FYEO-XRPL-03	High	Wrong ledger entry receives sponsor in MPToken escrow path	Remediated
FYEO-XRPL-04	High	removeEmptyHolding doesn't retrieve sponsor from trustline being deleted	Remediated
FYEO-XRPL-05	Medium	Payment with tfSponsorCreatedAccount does not validate reserve	Remediated
FYEO-XRPL-06	Low	Potential unsigned integer overflow in adjustOwnerCount	Remediated
FYEO-XRPL-07	Low	Trustline sponsor field default parameter is dangerous	Remediated
FYEO-XRPL-08	Low	sfSponsorFlags allowed without sfSponsor field	Remediated
FYEO-XRPL-09	Informational	Code Quality Concerns	Remediated
FYEO-XRPL-10	Informational	NFTokenUtils removeToken has unused sponsor variable	Remediated

Table 2: Findings Overview

The Classification of vulnerabilities

Security vulnerabilities and areas for improvement are weighted into one of several categories using, but is not limited to, the criteria listed below:

Critical – vulnerability will lead to a loss of protected assets

- This is a vulnerability that would lead to immediate loss of protected assets
- The complexity to exploit is low
- The probability of exploit is high

High - vulnerability has potential to lead to a loss of protected assets

- All discrepancies found where there is a security claim made in the documentation that cannot be found in the code
- All mismatches from the stated and actual functionality
- Unprotected key material
- Weak encryption of keys
- Badly generated key materials
- Txn signatures not verified
- Spending of funds through logic errors
- Calculation errors overflows and underflows

Medium - vulnerability hampers the uptime of the system or can lead to other problems

- Insecure calls to third party libraries
- Use of untested or nonstandard or non-peer-reviewed crypto functions
- Program crashes, leaves core dumps or writes sensitive data to log files

Low – vulnerability has a security impact but does not directly affect the protected assets

- Overly complex functions
- Unchecked return values from 3rd party libraries that could alter the execution flow

Informational

- General recommendations

Technical Analysis

The source code has been manually validated to the extent that the state of the repository allowed. The validation includes confirming that the code correctly implements the intended functionality.

Conclusion

Based on our review process, we conclude that the code implements the documented functionality to the extent of the reviewed code.

Technical Findings

General Observations

The update introduces a comprehensive sponsorship system to the XRP Ledger that allows one account (the sponsor) to cover the reserve requirements and transaction fees for another account (the sponsee). The feature adds two new transaction types: SponsorshipSet for creating and managing sponsorship arrangements, and SponsorshipTransfer for transferring existing sponsorships between accounts. Sponsors can choose to cover either reserve requirements (the XRP locked up when creating ledger objects like trust lines, offers, or tokens) or transaction fees, or both. The system supports both pre-funded sponsorships where reserve capacity is allocated upfront, and co-signed sponsorships where the sponsor must sign transactions in real-time to authorize the use of their reserves.

The implementation touches nearly every transaction type on the ledger, adding sponsor tracking to account roots, trust lines, and other ledger objects through new fields like sfSponsor, sfSponsoredOwnerCount, and sfSponsoringOwnerCount. Accounts can now have their reserve requirements calculated based on which objects are sponsored versus self-funded, enabling more flexible account management. The feature includes safety mechanisms such as permission flags (like asfDisallowIncomingSponsor) to control incoming sponsorship requests, and signature requirements (lsfSponsorshipRequireSignForFee and lsfSponsorshipRequireSignForReserve) to ensure sponsors maintain control over their commitments. This opens up new use cases like service providers covering costs for their users, or enabling accounts with minimal XRP balances to participate more fully in the network.

SponsorshipSet uses sfFeeAmount without presence check

Finding ID: FYEO-XRPL-01
Severity: **High**
Status: **Remediated**

Description

Both `SponsorshipSet::doApply()` and `SponsorshipSet::deleteSponsorship()` unconditionally access `sfFeeAmount` using `getFieldAmount()` when deleting a sponsorship. However, `sfFeeAmount` is defined as `soeOPTIONAL` in the `ltSPONSORSHIP` ledger format and is explicitly removed when depleted to zero (per `Transactor.cpp`). This causes a runtime failure when attempting to delete a sponsorship whose fee amount has been fully consumed.

Proof of Issue

File: src/xrpld/app/tx/detail/SponsorshipSet.cpp
Lines: doApply and deleteSponsorship

Both locations unconditionally call `getFieldAmount(sfFeeAmount)` without checking field presence:

```
auto const feeAmount = sponsorObjSle->getFieldAmount(sfFeeAmount);
(*sponsorAccSle)[sfBalance] += feeAmount;
```

Severity and Impact Summary

Runtime failure when deleting sponsorships with depleted fee amounts. Blocks legitimate cleanup operations and could prevent account deletion or trap funds. Requires specific sequence (create with fee, deplete to zero, delete) but represents normal operational flow.

Recommendation

Check field presence before accessing `sfFeeAmount` at both locations.

SponsorshipTransfer hardcodes sfSponsor breaking trustlines

Finding ID: FYEO-XRPL-02
Severity: **High**
Status: **Remediated**

Description

In `SponsorshipTransfer::doApply()` dissolve path, code uses hardcoded `sfSponsor` field instead of `sponsorField` variable. This breaks trustlines which must use `sfHighSponsor`/`sfLowSponsor`.

Proof of Issue

File name: src/xrpld/app/tx/detail/SponsorshipTransfer.cpp
Line number: 348

```
auto const oldSponsor = objSle->getAccountID(sfSponsor);
```

Severity and Impact Summary

Breaks trustline sponsorship transfer functionality.

Recommendation

Always use `sponsorField` variable, never hardcode `sfSponsor`.

Wrong ledger entry receives sponsor in MPToken escrow path

Finding ID: FYEO-XRPL-03
Severity: **High**
Status: **Remediated**

Description

When creating an MPToken during escrow finish, the sponsor is added to the destination account instead of the MPToken ledger entry. The `createMPToken` function creates an MPToken ledger entry (ltMPTOKEN) but doesn't handle sponsorship. The code then calls `addSponsorToLedgerEntry(sleDest, sponsor)` where `sleDest` is the destination account (ltACCOUNT_ROOT), not the MPToken. This adds the sponsor to the account instead of to the MPToken that was just created. The MPToken remains unsponsored, but the account gets a sponsor field which may be incorrect if the account already has a different sponsor.

Proof of Issue

File name: src/xrpld/app/tx/detail/Escrow.cpp
Line number: 838

```
if (!view.exists(keylet::mptoken(issuanceKey.key, receiver)) && createAsset &&
!receiverIssuer)
{
    auto const sponsor = getTxReserveSponsor(view, tx);
    if (auto const ret = checkInsufficientReserve(view, tx, sleDest, xrpBalance,
sponsor, 1); !isTesSuccess(ret))
        return ret;

    if (auto const ter = MPTokenAuthorize::createMPToken(view, mptID, receiver, 0);
!isTesSuccess(ter))
    {
        return ter;  // LCOV_EXCL_LINE
    }

    // update owner count.
    adjustOwnerCount(view, tx, sleDest, sponsor, 1, journal);
    addSponsorToLedgerEntry(sleDest, sponsor);  // sleDest is the account, not the
MPToken
}
```

Severity and Impact Summary

MPToken is created without the `sponsor` field, so when deleted, the sponsor won't be properly credited. The `Account` may incorrectly get a sponsor field added. Reserve accounting will be incorrect, leading to permanent state corruption.

Recommendation

Retrieve the MPToken SLE after creation and add the sponsor to it instead of to the account. Alternatively, modify `createMPToken` to accept a sponsor parameter and handle sponsorship internally.

removeEmptyHolding doesn't retrieve sponsor from trustline being deleted

Finding ID: FYEO-XRPL-04
Severity: **High**
Status: **Remediated**

Description

When removing an empty trustline in `removeEmptyHolding`, the code passes an empty optional `{}` as the sponsor parameter to `adjustOwnerCount`. This means it doesn't retrieve the actual sponsor from the trustline's `sfLowSponsor` or `sfHighSponsor` fields before deletion. If the trustline was sponsored, this will not decrement the sponsor's `sfSponsoringOwnerCount`, not increment the sponsor's reserve capacity back, and leave the sponsorship accounting in an inconsistent state.

Proof of Issue

File name: src/libxrpl/ledger/View.cpp
Line number: 1859

```
// Clear reserve for low account.
auto sleLowAccount = view.peek(keylet::account(line->at(sfLowLimit)->getIssuer()));
if (!sleLowAccount)
    return tecINTERNAL;

adjustOwnerCount(view, sleLowAccount, {}, -1, journal);
```

Severity and Impact Summary

Breaks sponsorship accounting when deleting sponsored trustlines, potentially locking sponsor reserves permanently. This is a state corruption issue, once a transaction is committed to the ledger, the incorrect accounting cannot be fixed. Sponsors could permanently lose reserve capacity.

Recommendation

Retrieve the sponsor from the trustline's `sfLowSponsor` or `sfHighSponsor` fields before calling `adjustOwnerCount`, similar to the pattern used elsewhere in the codebase (`View.cpp` in `rippleCreditIOU`).

Payment with tfSponsorCreatedAccount does not validate reserve

Finding ID: FYEO-XRPL-05
Severity: Medium
Status: Remediated

Description

When creating an account via Payment transaction with the `tfSponsorCreatedAccount` flag, the code in `preclaim` skips reserve validation entirely, assuming the sponsor will cover it. However, there is NO validation in either `preclaim` or `doApply` to ensure the sponsor account actually has sufficient reserve to sponsor the new account. This allows transactions to succeed in `preclaim` but potentially fail in `doApply` when the sponsor's reserve is insufficient, or worse, create accounts with insufficient sponsor reserves.

Proof of Issue

File name: src/xrpld/app/tx/detail/Payment.cpp
Line numbers: 302, 399

```
else if (txFlags & tfSponsorCreatedAccount)
{
    // The minimum amount when creating a Sponsored Account is 1 drop.
    // Since the reserve is covered by the sponsor, you don't need to hold the
1-increment reserve yourself.
}

if (txFlags & tfSponsorCreatedAccount)
{
    auto const sponsor = view().peek(keylet::account(account_));
    if (!sponsor)
        return tefINTERNAL;  // LCOV_EXCL_LINE
    auto const currentSponsoringAccountCount =
sponsor->getFieldU32(sfSponsoringAccountCount);
    sponsor->setFieldU32(sfSponsoringAccountCount, currentSponsoringAccountCount + 1);

    addSponsorToLedgerEntry(sleDst, sponsor);
    view().update(sponsor);
}
```

There is no call to `checkInsufficientReserve` to validate the sponsor has enough reserve to sponsor an account (which requires incrementing `sfSponsoringAccountCount`).

Severity and Impact Summary

This allows account creation to proceed without validating the sponsor has sufficient reserves. This can lead to Sponsor accounts being put into a state where their reserve requirement exceeds their balance.

The pattern used elsewhere in the codebase (CreateCheck.cpp, CreateOffer.cpp) always calls `checkInsufficientReserve` before creating sponsored objects, but Payment with `tfSponsorCreatedAccount` skips this validation.

Recommendation

Add reserve validation in `preclaim` for the sponsor account when `tfSponsorCreatedAccount` flag is set.

Potential unsigned integer overflow in adjustOwnerCount

Finding ID: FYEO-XRPL-06
Severity: Low
Status: Remediated

Description

In `adjustOwnerCount`, when `amount` is negative (indicating a “payback” operation), the expression `currentReserveCount - amount` performs unsigned arithmetic without overflow protection. When subtracting a negative amount, it effectively adds to `currentReserveCount`, but there’s no check to prevent the result from exceeding `UINT32_MAX`. Unlike the positive amount case which has an assertion protecting against underflow, there’s no corresponding check for overflow when `amount < 0`.

Proof of Issue

File name: src/libxrpl/ledger/View.cpp
Line number: 1176

```
auto const currentReserveCount = sponsorObjSle->getFieldU32(sfReserveCount);
if (amount > 0)
    XRPL_ASSERT(currentReserveCount >= amount, "xrpl::adjustOwnerCount : enough reserve
count");

if (currentReserveCount - amount > 0)
    sponsorObjSle->setFieldU32(sfReserveCount, currentReserveCount - amount);
else
    sponsorObjSle->makeFieldAbsent(sfReserveCount);
```

Severity and Impact Summary

When `amount` is negative and `currentReserveCount + |amount|` exceeds `UINT32_MAX`, the addition wraps around and an incorrect wrapped value gets written to `sfReserveCount`. This creates a correctness issue where reserve counts become corrupted. The likelihood depends on whether `amount` can realistically be a large negative value in practice.

Recommendation

Add overflow protection for negative amounts, similar to the existing underflow protection for positive amounts. Consider using signed arithmetic or explicit overflow checks before performing the operation.

Trustline sponsor field default parameter is dangerous

Finding ID: FYEO-XRPL-07
Severity: Low
Status: Remediated

Description

The `getLedgerEntryReserveSponsor()` has a default parameter `field = sfSponsor`, but trustlines must use `sfHighSponsor`/`sfLowSponsor`. The default makes it easy to accidentally use the wrong field on trustlines, silently ignoring correct sponsor fields.

Proof of Issue

File name: include/xrpl/ledger/View.h
Line number: 453

```
std::optional<AccountID>
getLedgerEntryReserveSponsorAccountID(std::shared_ptr<SLE const> const& sle, SF_ACCOUNT
const& field = sfSponsor);
```

Severity and Impact Summary

Future regression concern. Trustlines explicitly require different sponsor fields but default encourages wrong usage.

Recommendation

Remove the default parameter to force explicit field specification at all call sites. Consider adding additional checks.

sfSponsorFlags allowed without sfSponsor field

Finding ID: FYEO-XRPL-08
Severity: Low
Status: Remediated

Description

Preflight blocks `sfSponsorSignature` without `sfSponsor`, but does not block `sfSponsorFlags` without `sfSponsor`. This allows malformed transactions where sponsor flags exist but no sponsor is specified, potentially leading to undefined behavior in later logic that reads flags assuming sponsor exists.

Proof of Issue

File name: src/xrpld/app/tx/detail/Transactor.cpp
Line number: 161

```
if (hasSponsorSig && !hasSponsor)
    return temMALFORMED;
// Missing: if (hasSponsorFlags && !hasSponsor)
```

Severity and Impact Summary

Could lead to malformed transactions entering later logic stages.

Recommendation

Block sponsor flags if no sponsor is present.

Code Quality Concerns

Finding ID: FYEO-XRPL-09
Severity: **Informational**
Status: **Remediated**

Description

Variable shadowing occurs where a nested `sponsor` variable (`std::shared_ptr<SLE const>`) shadows an outer `sponsor` variable (`std::optional<AccountID>`). While functionally correct due to scoping, this reduces code clarity and could cause confusion during maintenance.
A redundant `XRPL_ASSERT(sponsorSle, ...)` appears immediately inside an `if (sponsorSle)` block. The assertion can never fail since the condition already verifies `sponsorSle` is truthy.
The function `getTxReserveSponsor` has a redundant `const` qualifier on its return type that applies to the returned `shared_ptr` value itself, making it ineffective.

Proof of Issue

File: src/xrpld/app/tx/detail/NFTokenAcceptOffer.cpp
Lines: 337 and 358

Line 337 declares `sponsor` as `std::optional<AccountID>`, then line 358 redeclares `sponsor` in a nested scope as `std::shared_ptr<SLE const>`, shadowing the outer variable.

File: src/libxrpl/ledger/View.cpp
Lines: 1140,1142

The assertion appears directly after the conditional check that guarantees the same condition.

File: include/xrpl/ledger/View.h
Line: 453

The return type `std::shared_ptr<SLE const> const` has a redundant second `const` qualifier on the value being returned.

Severity and Impact Summary

Minor code quality issues that reduce readability. No functional impact.

Recommendation

Rename the inner variable to eliminate shadowing.
Remove the redundant assertion to improve code clarity.
Remove the redundant `const` qualifier from the return type.

NFTokenUtils removeToken has unused sponsor variable

Finding ID: FYEO-XRPL-10
Severity: **Informational**
Status: **Remediated**

Description

In `removeToken()`, local variable `sponsor` is declared but never used. The function calls `getLedgerEntryReserveSponsor()` twice instead of reusing the local variable. This signals copy-paste error and invites future divergence.

Proof of Issue

File name: src/xrpld/app/tx/detail/NFTokenUtils.cpp
Line number: 489

```
auto const sponsor = getLedgerEntryReserveSponsor(view, curr);
adjustOwnerCount(
    view,
    view.peek(keylet::account(owner)),
    getLedgerEntryReserveSponsor(view, curr),  // Called again instead of using sponsor
    -1,
    ...
);
```

Severity and Impact Summary

Potential copy-paste error. Not a functional bug but maintenance risk.

Recommendation

Use the local `sponsor` variable in adjustOwnerCount call.

Our Process

Methodology

FYEO Inc. uses the following high-level methodology when approaching engagements. They are broken up into the following phases.

Figure 2: Methodology Flow

Kickoff

The project is kicked off as the sales process has concluded. We typically set up a kickoff meeting where project stakeholders are gathered to discuss the project as well as the responsibilities of participants. During this meeting we verify the scope of the engagement and discuss the project activities. It's an opportunity for both sides to ask questions and get to know each other. By the end of the kickoff there is an understanding of the following:

- Designated points of contact
- Communication methods and frequency
- Shared documentation
- Code and/or any other artifacts necessary for project success
- Follow-up meeting schedule, such as a technical walkthrough
- Understanding of timeline and duration

Ramp-up

Ramp-up consists of the activities necessary to gain proficiency on the project. This can include the steps needed for familiarity with the codebase or technological innovation utilized. This may include, but is not limited to:

- Reviewing previous work in the area including academic papers
- Reviewing programming language constructs for specific languages
- Researching common flaws and recent technological advancements

Review

The review phase is where most of the work on the engagement is completed. This is the phase where we analyze the project for flaws and issues that impact the security posture. Depending on the project this may include an analysis of the architecture, a review of the code, and a specification matching to match the architecture to the implemented code.

In this code audit, we performed the following tasks:

1. Security analysis and architecture review of the original protocol
2. Review of the code written for the project
3. Compliance of the code with the provided technical documentation

The review for this project was performed using manual methods and utilizing the experience of the reviewer. No dynamic testing was performed, only the use of custom-built scripts and tools were used to assist the reviewer during the testing. We discuss our methodology in more detail in the following sections.

Code Safety

We analyzed the provided code, checking for issues related to the following categories:

- General code safety and susceptibility to known issues
- Poor coding practices and unsafe behavior
- Leakage of secrets or other sensitive data through memory mismanagement
- Susceptibility to misuse and system errors
- Error management and logging

This list is general and not comprehensive, meant only to give an understanding of the issues we are looking for.

Technical Specification Matching

We analyzed the provided documentation and checked that the code matches the specification. We checked for things such as:

- Proper implementation of the documented protocol phases
- Proper error handling
- Adherence to the protocol logical description

Reporting

FYEO Inc. delivers a draft report that contains an executive summary, technical details, and observations about the project.

The executive summary contains an overview of the engagement including the number of findings as well as a statement about our general risk assessment of the project. We may conclude that the overall risk is low but depending on what was assessed we may conclude that more scrutiny of the project is needed.

We report security issues identified, as well as informational findings for improvement, categorized by the following labels:

- Critical
- High
- Medium
- Low
- Informational

The technical details are aimed more at developers, describing the issues, the severity ranking and recommendations for mitigation.

As we perform the audit, we may identify issues that aren't security related, but are general best practices and steps that can be taken to lower the attack surface of the project. We will call those out as we encounter them and as time permits.

As an optional step, we can agree on the creation of a public report that can be shared and distributed with a larger audience.

Verify

After the preliminary findings have been delivered, this could be in the form of the approved communication channel or delivery of the draft report, we will verify any fixes within a window of time specified in the project. After the fixes have been verified, we will change the status of the finding in the report from open to remediated.

The output of this phase will be a final report with any mitigated findings noted.

Additional Note

It is important to note that, although we did our best in our analysis, no code audit or assessment is a guarantee of the absence of flaws. Our effort was constrained by resource and time limits along with the scope of the agreement.

While assessing the severity of the findings, we considered the impact, ease of exploitability, and the probability of attack. This is a solid baseline for severity determination.